Open Influence Diagrams: Proofs

Simon Frost

2026-10-16

Contents

Boundary observability **1**

Boundary observability

```
import CategoricalBayesianNetworksProofs
import InfluenceDiagramsProofs
```

The first module of the proofs of `CategoricalInfluenceDiagrams.jl` (ADR 0018, decision 5). It imports the two proof projects this one builds on: the typed open-network syntax of `CategoricalBayesianNetworks.jl` (ADR 0010, namespace `OpenNet`) and the finite influence-diagram model of `InfluenceDiagrams.jl` (namespace `InfluenceDiagramsProofs`).

It states the one piece of structure that open influence diagrams add to the boundary of an open network: every boundary variable is **observed**, remembered information that a later decision may read, or **hidden**. An interface with observability is an `OpenNet.Interface` with a flag on every port, and the tensor of two such interfaces places ports and flags side by side. **Information locality**, rule 3 of the typed-interface rule, says that no decision of a finite influence diagram reads a variable that its boundary declares hidden.

Decision and utility generators on the typed syntax, the expectation-semiring kernel category, the functoriality of the fixed-strategy interpretation, the contextual identity and the isolated-optimisation obstruction are later milestones (M4 of the plan); nothing here states them.

```
set_option autoImplicit false

namespace OpenInfluence

/-- The observability of a boundary variable (ADR 0018, decision 2). -/
inductive Observability where
  /-- Remembered information that a later decision may read. -/
  | observed
  /-- State that crosses the boundary but that no decision on the far side may read. -/
  | hidden
  deriving DecidableEq, Repr

/-- An interface of the open-network syntax with an observability flag on every port. -/
structure ObservedInterface (S : OpenNet.Signature) where
  /-- The underlying finite typed interface. -/
  iface : OpenNet.Interface S
  /-- The flag of each port. -/
  flag : iface.Port → Observability

namespace ObservedInterface

variable {S : OpenNet.Signature}

/-- The tensor of two interfaces with observability: ports and flags side by side. -/
def sum (X Y : ObservedInterface S) : ObservedInterface S :=
  ⟨X.iface.sum Y.iface, Sum.elim X.flag Y.flag⟩
```

```
/-- The empty interface, the unit of `sum`. -/
def empty (S : OpenNet.Signature) : ObservedInterface S :=
  ⟨OpenNet.Interface.empty S, Empty.elim⟩

/-- The tensor keeps the flags of the left interface. -/
theorem sum_flag_inl (X Y : ObservedInterface S) (p : X.iface.Port) :
    (X.sum Y).flag (Sum.inl p) = X.flag p := rfl

/-- The tensor keeps the flags of the right interface. -/
theorem sum_flag_inr (X Y : ObservedInterface S) (p : Y.iface.Port) :
    (X.sum Y).flag (Sum.inr p) = Y.flag p := rfl

end ObservedInterface

open InfluenceDiagramsProofs

/-- Information locality (rule 3 of ADR 0018, decision 2): given the observability of the
boundary variables of a finite influence diagram (`none` for an internal variable), no
decision has a hidden boundary variable in its information set. -/
def InformationLocal (D : FinInfluenceDiagram) (boundary : D.V → Option Observability) :
    Prop :=
  ∀ d v, v ∈ D.info d → boundary v ≠ some .hidden

/-- A diagram whose boundary declares nothing hidden is information-local. -/
theorem informationLocal_of_no_hidden (D : FinInfluenceDiagram)
    (boundary : D.V → Option Observability) (h : ∀ v, boundary v ≠ some .hidden) :
    InformationLocal D boundary :=
  fun _ v _ ⇒ h v

/-- Revealing boundary variables (turning hidden flags into observed or internal ones)
preserves information locality. -/
theorem InformationLocal.of_reveal {D : FinInfluenceDiagram}
    {b b' : D.V → Option Observability}
    (h : InformationLocal D b) (hb : ∀ v, b' v = some .hidden → b v = some .hidden) :
    InformationLocal D b' :=
  fun d v hv hid ⇒ h d v hv (hb v hid)

end OpenInfluence
```